

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Model 2026 - Informatică și Tehnologia Informației

Cuprins

SUBIECTUL I (60 de puncte)	2
1. Structura de date: Listă simplu înălțuită	2
a) Noțiuni preliminare: Parcurgerea nodurilor	2
b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare	2
MODUL 1: Alocare Dinamică a Memoriei (Pointeri)	2
MODUL 2: Alocare Statică a Memoriei (Tablouri/Vectori de Noduri)	4
c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)	6
METODA A: Implementare prin Alocare Dinamică a Memoriei	6
METODA B: Implementare prin Alocare Statică (Tablouri)	8
2. Sisteme de operare, rețele și securitate	9
a) Noțiuni preliminare:	9
b) Trei tipuri de amenințări informatice:	9
c) Două acțiuni ale unui antivirus la detectarea unui virus:	10
d) Exemplu de program antivirus cunoscut:	10
3. Programare: Generare etichete sală spectacol	10
METODA 1: Rezolvare prin Metode Matematice (Numerice)	10
METODA 2: Rezolvare prin Conversie la Șiruri de Caractere (String-uri)	12
4. Baze de date: Asociație Festivaluri	13
a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)	13
b) Modelul Fizic (Structura tabelor)	14
c) Adăugarea datelor de bază (Comenzi SQL)	15
d) Alternativă: Procedura detaliată pas cu pas într-un SGBD vizual (MySQL Workbench)	15
e) Interogări SQL pentru extragerea informațiilor de sinteză	15
SUBIECTUL al II-lea (30 de puncte)	16
1. Proiectarea unei activități didactice (Proiecte complete de Lecție)	16
PLAN DIDACTIC PENTRU SECVENȚA A (Divide et Impera)	16
Scenariul Didactic pe Etape (ERRE)	16
PLAN DIDACTIC PENTRU SECVENȚA B (Dispozitive de stocare)	17
Scenariul Didactic pe Etape (ERRE)	17
2. Evaluarea rezultatelor școlare: Proba Scrisă	18
a) Avantaje și limite ale probei scrise:	18
b) Elaborarea itemilor de evaluare	18
PENTRU SECVENȚA A (Subprograme / Divide et Impera)	18
PENTRU SECVENȚA B (Dispozitive de stocare)	19

SUBIECTUL I (60 de puncte)

1. Structura de date: Listă simplu înlănțuită

a) Noțiuni preliminare: Parcurgerea nodurilor

O listă simplu înlănțuită este o structură de date dinamică formată din noduri alocate necontiguu în memoria Heap (în cazul alocării dinamice) sau stocate în zone contigue de memorie (în cazul alocării statice). Fiecare nod conține cel puțin două câmpuri:

- **Informația utilă** (datele propriu-zise).
- **Legătura** către nodul următor (adresa în cazul alocării dinamice, sau indicele în cazul celei statice).

Parcurgerea nodurilor: Pornind de la primul nod al listei (`prim`), se vizitează nodul curent și se avansează la succesorul său prin reatribuirea pointerului/indicelui de parcurgere cu legătura sa: `p = p->urm` (alocare dinamică) sau `p = lista[p].urm` (alocare statică), până când legătura devine indicator de sfârșit de listă (`NULL`, `nil` sau `-1`).

—

b) Descrierea și exemplificarea etapelor pentru cele 4 operații de inserare

Presupunem o listă cu 5 noduri având valorile inițiale: `[10] -> [20] -> [30] -> [40] -> [50]`. Noul nod de inserat are valoarea 99. Prezentăm ambele moduri de alocare cerute de programă.

MODUL 1: Alocare Dinamică a Memoriei (Pointeri)

Definirea structurilor: **C++:**

```
struct Nod {
    int info;
    Nod* urm;
};
Nod* prim; // Adresa primului nod
```

Pascal:

```
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
var
    prim: PNod;
```

Cele 4 operații de inserare specifice:

1. Inserarea înainte de primul nod (la începutul listei)

- **Etape:** Se alocă dinamic noul nod nou cu valoarea 99. Legătura noului nod preia adresa primului nod actual (`nou->urm = prim`). Pointerul de start `prim` devine adresa lui nou.

• Cod C++:

```
Nod* nou = new Nod{99, prim};
prim = nou;
```

• Cod Pascal:

```
new(nou); nou^.info := 99;
nou^.urm := prim;
prim := nou;
```

2. Inserarea după ultimul nod (la sfârșitul listei)

- **Etape:** Se alocă noul nod nou și se setează legătura lui pe nullptr / nil. Se parcurge lista pentru a identifica ultimul nod p (cel pentru care p->urm == nullptr). Legătura lui p este setată să indice către nou.

- **Cod C++:**

```
Nod* nou = new Nod{99, nullptr};
if (prim == nullptr) prim = nou;
else {
    Nod* p = prim;
    while (p->urm != nullptr) p = p->urm;
    p->urm = nou;
}
```

- **Cod Pascal:**

```
new(nou); nou^.info := 99; nou^.urm := nil;
if prim = nil then prim := nou
else begin
    p := prim;
    while p^.urm <> nil do p := p^.urm;
    p^.urm := nou;
end;
```

3. Inserarea înainte de un nod intermediar dat (ex. înaintea nodului cu valoarea 30)

- **Etape:** Se alocă nou cu valoarea 99. Se parcurge lista cu un pointer ant pentru a găsi nodul aflat înaintea celui căutat (nodul cu valoarea 20). Legătura lui nou devine succesorul lui ant (nou->urm = ant->urm). Legătura lui ant devine nou.

- **Cod C++:**

```
Nod* nou = new Nod{99, nullptr};
Nod* ant = prim;
while (ant->urm != nullptr && ant->urm->info != 30) ant = ant->urm;
nou->urm = ant->urm;
ant->urm = nou;
```

- **Cod Pascal:**

```
new(nou); nou^.info := 99;
ant := prim;
while (ant^.urm <> nil) and (ant^.urm^.info <> 30) do ant := ant^.urm;
nou^.urm := ant^.urm;
ant^.urm := nou;
```

4. Inserarea după un nod intermediar dat (ex. după nodul cu valoarea 30)

- **Etape:** Se alocă nou cu valoarea 99. Se parcurge lista pentru a identifica nodul dat p (cel cu valoarea 30). Legătura lui nou preia succesorul lui p (nou->urm = p->urm). Legătura lui p devine nou.

- **Cod C++:**

```
Nod* nou = new Nod{99, nullptr};
Nod* p = prim;
while (p != nullptr && p->info != 30) p = p->urm;
if (p != nullptr) {
```

```
nou->urm = p->urm;
p->urm = nou;
}
```

• **Cod Pascal:**

```
new(nou); nou^.info := 99;
p := prim;
while (p <> nil) and (p^.info <> 30) do p := p^.urm;
if p <> nil then begin
    nou^.urm := p^.urm;
    p^.urm := nou;
end;
```

MODUL 2: Alocare Statică a Memoriei (Tablouri/Vectori de Noduri)

În alocarea statică, memoria este rezervată la compilare sub forma unui tablou de înregistrări/structuri. Indicatorii de legătură sunt reprezentați prin indici ai tabloului, valoarea -1 marcând absența unui succesor (echivalentul NULL). Un vector suplimentar sau o listă de noduri libere (disponibil) ține evidența elementelor neutilizate.

Definirea structurilor: **C++:**

```
struct NodStatic {
    int info;
    int urm;
};
NodStatic memoria[100]; // Pool-ul static de memorie
int prim = -1;           // Începutul listei active
int disponibil = 0;      // Primul nod liber din pool (inițial înlănțuite 0->1->2...->99->-1)
```

Pascal:

```
type
    TNodStatic = record
        info: integer;
        urm: integer;
    end;
var
    memoria: array[0..99] of TNodStatic;
    prim: integer = -1;
    disponibil: integer = 0;
```

Funcție helper de alocare a unui nod din spațiul disponibil:

```
int alocaNod(int val) {
    int nou = disponibil;
    if (disponibil != -1) {
        disponibil = memoria[disponibil].urm;
        memoria[nou].info = val;
        memoria[nou].urm = -1;
    }
    return nou;
}
```

Cele 4 operații de inserare specifice în reprezentarea statică:

1. Inserarea înainte de primul nod

- **Etape:** Se obține indicele unui nod nou (nou) din pool. Legătura lui nou devine prim. Variabila prim ia valoarea nou.

- **C++:**

```
int nou = alocaNod(99);
memoria[nou].urm = prim;
prim = nou;
```

- **Pascal:**

```
nou := disponibil; disponibil := memoria[disponibil].urm;
memoria[nou].info := 99;
memoria[nou].urm := prim;
prim := nou;
```

2. Inserarea după ultimul nod

- **Etape:** Se obține nodul nou. Dacă lista este vidă, prim devine nou. Altfel, se parcurge vectorul pornind de la prim până la un element p pentru care memoria[p].urm == -1. Legătura memoria[p].urm devine nou.

- **C++:**

```
int nou = alocaNod(99);
if (prim == -1) prim = nou;
else {
    int p = prim;
    while (memoria[p].urm != -1) p = memoria[p].urm;
    memoria[p].urm = nou;
}
```

3. Inserarea înainte de un nod intermediar dat (ex. nodul cu valoarea 30)

- **Etape:** Se determină nodul anterior ant. Se leagă noul nod la următorul lui ant și se actualizează legătura lui ant către nou.

- **C++:**

```
int nou = alocaNod(99);
int ant = prim;
while (memoria[ant].urm != -1 && memoria[memoria[ant].urm].info != 30) {
    ant = memoria[ant].urm;
}
memoria[nou].urm = memoria[ant].urm;
memoria[ant].urm = nou;
```

4. Inserarea după un nod intermediar dat (ex. nodul cu valoarea 30)

- **Etape:** Se caută nodul p cu valoarea 30. Noul nod preia succesorul lui p, apoi p se leagă de nou.

- **C++:**

```
int nou = alocaNod(99);
int p = prim;
while (p != -1 && memoria[p].info != 30) p = memoria[p].urm;
if (p != -1) {
    memoria[nou].urm = memoria[p].urm;
    memoria[p].urm = nou;
}
```

—

c) Exemplu de utilizare într-o problemă completă (Inserare Ordonată)

- **Enunț:** Se citesc de la tastatură numere întregi nenule până la întâlnirea valorii 0. Să se construiască o listă simplu înlănțuită în care elementele să fie memorate în ordine strict crescătoare (inserare ordonată), iar la final să se afișeze elementele listei separate prin spațiu.

METODA A: Implementare prin Alocare Dinamică a Memoriei

- **Descrierea soluției:** Se pornește cu o listă vidă (prim = nullptr). Pentru fiecare valoare citită, se alocă un nod. Dacă lista este vidă sau valoarea este mai mică decât informația din primul nod, inserarea se face la început. Altfel, parcurgem lista cu un pointer ajutor până găsim locul corect (înaintea primului nod cu valoare mai mare sau la final) și efectuăm inserarea.

• **Cod C++:**

```
#include <iostream>
using namespace std;

struct Nod {
    int info;
    Nod* urm;
};

void inserareOrdonataDinamic(Nod* &prim, int val) {
    Nod* nou = new Nod{val, nullptr};
    if (prim == nullptr || prim->info >= val) {
        nou->urm = prim;
        prim = nou;
    } else {
        Nod* curent = prim;
        while (curent->urm != nullptr && curent->urm->info < val) {
            curent = curent->urm;
        }
        nou->urm = curent->urm;
        curent->urm = nou;
    }
}

void afisareDinamic(Nod* prim) {
    Nod* p = prim;
    while (p != nullptr) {
        cout << p->info << " ";
        p = p->urm;
    }
    cout << "\n";
}

int main() {
    Nod* prim = nullptr;
    int x;
    while (cin >> x && x != 0) {
        inserareOrdonataDinamic(prim, x);
    }
    afisareDinamic(prim);
}
```

```
    return 0;
}
```

• **Cod Pascal:**

```
program ListaOrdonataDinamica;
type
  PNode = ^TNode;
  TNode = record
    info: integer;
    urm: PNode;
  end;
var
  prim: PNode;
  x: integer;

procedure inserareOrdonata(var prim: PNode; val: integer);
var
  nou, curent: PNode;
begin
  new(nou);
  nou^.info := val;
  nou^.urm := nil;
  if (prim = nil) or (prim^.info >= val) then
  begin
    nou^.urm := prim;
    prim := nou;
  end
  else
  begin
    curent := prim;
    while (curent^.urm <> nil) and (curent^.urm^.info < val) do
      curent := curent^.urm;
    nou^.urm := curent^.urm;
    curent^.urm := nou;
  end;
end;

procedure afisare(p: PNode);
begin
  while p <> nil do
  begin
    write(p^.info, ' ');
    p := p^.urm;
  end;
  writeln;
end;

begin
  prim := nil;
  read(x);
  while x <> 0 do
  begin
    inserareOrdonata(prim, x);
    read(x);
  end;
end;
```

```
    afisare(prim);
end.
```

METODA B: Implementare prin Alocare Statică (Tablouri)

- **Descrierea soluției:** Utilizăm un tablou static de înregistrări. Gestionăm o listă secundară a nodurilor libere (disponibil). Algoritmul de inserare ordonată păstrează aceeași logică, dar înlocuiește referințele de adrese cu indici ai tabloului.

- **Cod C++:**

```
#include <iostream>
using namespace std;

struct NodStatic {
    int info;
    int urm;
};

NodStatic memoria[1000];
int prim = -1;
int disponibil = 0;

void initializareMemorie() {
    for (int i = 0; i < 999; ++i) {
        memoria[i].urm = i + 1;
    }
    memoria[999].urm = -1;
}

int alocaNodStatic(int val) {
    if (disponibil == -1) return -1; // Memorie plină
    int nou = disponibil;
    disponibil = memoria[disponibil].urm;
    memoria[nou].info = val;
    memoria[nou].urm = -1;
    return nou;
}

void inserareOrdonataStatic(int val) {
    int nou = alocaNodStatic(val);
    if (nou == -1) return;

    if (prim == -1 || memoria[prim].info >= val) {
        memoria[nou].urm = prim;
        prim = nou;
    } else {
        int curent = prim;
        while (memoria[curent].urm != -1 && memoria[memoria[curent].urm].info < val)
        {
            curent = memoria[curent].urm;
        }
        memoria[nou].urm = memoria[curent].urm;
        memoria[curent].urm = nou;
    }
}
```



```
void afisareStatic() {
    int p = prim;
    while (p != -1) {
        cout << memoria[p].info << " ";
        p = memoria[p].urm;
    }
    cout << "\n";
}

int main() {
    initializareMemorie();
    int x;
    while (cin >> x && x != 0) {
        inserareOrdonataStatic(x);
    }
    afisareStatic();
    return 0;
}
```

2. Sisteme de operare, rețele și securitate

a) Noțiuni preliminare:

1. **Sistem de calcul:** Ansamblul fizic (hardware: procesor, memorie, dispozitive I/O) și logic (software: sistem de operare, programe) care prelucrează automat datele.
2. **Sistem de operare:** Componenta software fundamentală care acționează ca interfață între utilizator și hardware-ul fizic, având rolul de a gestiona resursele sistemului (CPU, RAM, fișiere, periferice).
3. **Rețea de calculatoare:** Un ansamblu de sisteme de calcul interconectate prin medii de transmisie (cablu, Wi-Fi) cu scopul partajării de resurse hardware (imprimante) și software (fișiere, baze de date).

b) Trei tipuri de amenințări informatice:

1. **Troian (Trojan Horse):**
 - **Caracteristica 1:** Se ascunde în interiorul unui program aparent legitim sau util (joc, utilitar) pentru a înșela utilizatorul.
 - **Caracteristica 2:** Nu se autoreplică (nu infectează alte fișiere singur), ci deschide porturi (backdoors) pentru atacatori externi.
2. **Ransomware:**
 - **Caracteristica 1:** Criptează fișierele de date ale utilizatorului cu algoritmi puternici de criptare (ex. AES/RSA).
 - **Caracteristica 2:** Solicită plata unei răscumpărări (de regulă în criptomonede) pentru furnizarea cheii de decriptare.
3. **Vierme (Worm):**
 - **Caracteristica 1:** Se propagă autonom prin rețea, fără a avea nevoie de atașarea la un program gazdă.
 - **Caracteristica 2:** Exploatează vulnerabilități ale sistemelor de operare pentru a se replica masiv, consumând lățimea de bandă a rețelei.

c) Două acțiuni ale unui antivirus la detectarea unui virus:

1. **Carantinarea:** Mutarea fișierului infectat într-un director securizat și izolat, criptat, pentru a preveni execuția sa accidentală și răspândirea infecției.
2. **Dezinfectarea (Curățarea):** Eliminarea secvenței de cod malware din fișierul gazdă, restaurându-l în starea inițială, funcțională.

d) Exemplu de program antivirus cunoscut:

- Bitdefender Antivirus sau Kaspersky Anti-Virus.

—

3. Programare: Generare etichete sală spectacol

Avem de generat o matrice de dimensiuni $m \times n$ în care fiecare element $A[i][j]$ este cea mai mică etichetă ce se poate forma prin alipirea indicilor i (rând) și j (scaun), adică $\min(s + d, d + s)$ unde „+” reprezintă operația de alipire/concatenare. Rezultatul va fi scris în fișierul `def2025.txt`.

METODA 1: Rezolvare prin Metode Matematice (Numerice)

C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>

using namespace std;

// Subprogramul cerut: alipește cifrele lui d la dreapta lui s
void eticheta(int s, int d, int &n) {
    int temp = d;
    int p = 1;
    while (temp > 0) {
        p *= 10;
        temp /= 10;
    }
    n = s * p + d;
}

int main() {
    int m, n;
    if (!(cin >> m >> n)) return 0;

    int A[101][101];
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            int val1, val2;
            eticheta(j, i, val1); // scaunul j urmat de rândul i
            eticheta(i, j, val2); // rândul i urmat de scaunul j
            A[i][j] = min(val1, val2);
        }
    }

    ofstream fout("def2025.txt");
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            fout << A[i][j] << (j == n ? "" : " ");
        }
    }
```

```

        fout << "\n";
    }
    fout.close();

    return 0;
}

```

Pascal:

```

program GenerareEticheteMatematic;
uses math;

var
    m, n, i, j, val1, val2: integer;
    A: array[1..100, 1..100] of integer;
    fout: text;

procedure eticheta(s, d: integer; var n: integer);
var
    temp, p: integer;
begin
    temp := d;
    p := 1;
    while temp > 0 do
    begin
        p := p * 10;
        temp := temp div 10;
    end;
    n := s * p + d;
end;

begin
    readln(m, n);

    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            eticheta(j, i, val1);
            eticheta(i, j, val2);
            A[i, j] := min(val1, val2);
        end;
    end;

    assign(fout, 'def2025.txt');
    rewrite(fout);
    for i := 1 to m do
    begin
        for j := 1 to n do
        begin
            write(fout, A[i, j]);
            if j < n then
                write(fout, ' ');
            end;
            writeln(fout);
        end;
    end;
end;

```

```
close(fout);
end.
```

METODA 2: Rezolvare prin Conversie la Șiruri de Caractere (String-uri)

C++ (folosind std::to_string și std::stoi):

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>

using namespace std;

void etichetaString(int s, int d, int &n) {
    string str_s = to_string(s);
    string str_d = to_string(d);
    n = stoi(str_s + str_d);
}

int main() {
    int m, n;
    if (!(cin >> m >> n)) return 0;

    int A[101][101];
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            int val1, val2;
            etichetaString(j, i, val1);
            etichetaString(i, j, val2);
            A[i][j] = min(val1, val2);
        }
    }

    ofstream fout("def2025.txt");
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            fout << A[i][j] << (j == n ? " " : " ");
        }
        fout << "\n";
    }
    fout.close();
    return 0;
}
```

Pascal (folosind procedurile standard Str și Val):

```
program GenerareEticheteString;
uses math;

var
    m, n, i, j, val1, val2: integer;
    A: array[1..100, 1..100] of integer;
    fout: text;

procedure etichetaString(s, d: integer; var n: integer);
var
    str_s, str_d, str_n: string;
```

```

code: integer;
begin
  str(s, str_s);
  str(d, str_d);
  str_n := str_s + str_d;
  val(str_n, n, code);
end;

begin
  readln(m, n);

  for i := 1 to m do
  begin
    for j := 1 to n do
    begin
      etichetaString(j, i, val1);
      etichetaString(i, j, val2);
      A[i, j] := min(val1, val2);
    end;
  end;

  assign(fout, 'def2025.txt');
  rewrite(fout);
  for i := 1 to m do
  begin
    for j := 1 to n do
    begin
      write(fout, A[i, j]);
      if j < n then
        write(fout, ' ');
      end;
      writeln(fout);
    end;
  end;
  close(fout);
end.

```

4. Baze de date: Asociație Festivaluri

a) Modelul conceptual (Entități, Atribute, Relații, Forme Normale, Restricții)

Entități și atribute:

1. FESTIVAL: id_festival (PK - întreg), denumire (text, UNIQUE, NOT NULL), tematica (text), site_oficial (text, poate fi NULL).
2. TIP_EVENT: id_tip (PK - întreg), denumire_tip (text, UNIQUE, NOT NULL - ex. concert, workshop), spatiu_necesar (text), nr_max_participanti (întreg), varsta_minima (întreg).
3. EVENIMENT: id_eveniment (PK - întreg), id_festival (FK - întreg), id_tip (FK - întreg), denumire (text), descriere (text), data_ora_inceput (datetime), durata (întreg, minute), locatie (text), categorie_public_tinta (text).

Relații:

- Relația FESTIVAL - EVENIMENT: o relație de tip 1:M (Un festival poate cuprinde mai multe evenimente, un eveniment aparține unui singur festival).

- Relația TIP_EVENTIMENT - EVENTIMENT: o relație de tip 1:M (Un tip de eveniment poate fi asociat mai multor evenimente concrete, un eveniment are un singur tip).

Forme Normale:

- **1NF:** Toate atributele au doar valori atomice (ex. nu există liste de evenimente în tabela FESTIVAL).
- **2NF:** Baza este în 1NF și toate atributele non-cheie depind funcțional complet de cheia primară (cheile primare sunt simple, formate dintr-un singur câmp).
- **3NF:** Baza este în 2NF și nu conține dependențe tranzitive (atributele din EVENTIMENT depind doar de id_eveniment, nu tranzitiv prin alte câmpuri).

Restricții:

- durata > 0 în tabela EVENTIMENT.
- id_festival și id_tip în EVENTIMENT sunt FOREIGN KEY cu clauza ON DELETE CASCADE sau RESTRICT.

b) Modelul Fizic (Structura tabelelor)

Tabelă	Câmp	Tip de date	Restricții / Rol
FESTIVAL	id_festival	INT	PRIMARY KEY, AUTO_INCREMENT
	denumire	VARCHAR(100)	UNIQUE, NOT NULL
	tematica	VARCHAR(255)	
	site_oficial	VARCHAR(100)	NULL
TIP_EVENTIMENT	id_tip	INT	PRIMARY KEY, AUTO_INCREMENT
	denumire_tip	VARCHAR(50)	UNIQUE, NOT NULL
	spatiu_necesar	VARCHAR(100)	
	nr_max_participanti	INT	CHECK(nr_max_participanti > 0)
EVENTIMENT	varsta_minima	INT	CHECK(varsta_minima >= 0)
	id_eveniment	INT	PRIMARY KEY, AUTO_INCREMENT
	id_festival	INT	FOREIGN KEY REFERENCES FESTIVAL(id_festival)
	id_tip	INT	FOREIGN KEY REFERENCES TIP_EVENTIMENT(id_tip)
	denumire	VARCHAR(100)	NOT NULL
	descriere	TEXT	NULL
	data_ora_inceput	DATETIME	NOT NULL
	durata	INT	CHECK(durata > 0)
	locatie	VARCHAR(100)	
	categorie_public_tinut	VARCHAR(50)	

c) Adăugarea datelor de bază (Comenzi SQL)

Pentru a adăuga un nou festival cu numele „EduCode 2025” și tematica „Inovatie in predarea informaticii”:

```
INSERT INTO FESTIVAL (denumire, tematica, site_oficial)
VALUES ('EduCode 2025', 'Inovatie in predarea informaticii', NULL);
```

d) Alternativă: Procedura detaliată pas cu pas într-un SGBD vizual (MySQL Workbench)

- Conectare:** Deschideți aplicația MySQL Workbench și conectați-vă la instanța locală a serverului SQL.
- Creare Schemă:** Executați comanda `CREATE DATABASE festivaluri_db;` sau faceți click dreapta în panoul stâng (Schemas) pe „Create Schema”, introduceți numele bazei și apăsați „Apply”.
- Creare Tabele prin Interfață:**
 - Dublu click pe schema nou creată pentru a o activa.
 - Click dreapta pe „Tables” -> „Create Table...”.
 - Pentru tabela `FESTIVAL`: adăugați câmpurile `id_festival` (bifați PK, NN, AI), `denumire` (`VARCHAR(100)`, NN, UQ), `tematica` (`VARCHAR(255)`) și `site_oficial`. Apăsați „Apply”.
 - Repetati procesul pentru `TIP_EVENTIMENT` și `EVENTIMENT`. În fila „Foreign Keys” a tabeli `EVENTIMENT`, asociați `id_festival` cu tabela `FESTIVAL` și `id_tip` cu tabela `TIP_EVENTIMENT`.
- Adăugare Date:**
 - Faceți click dreapta pe tabela `FESTIVAL` și selectați „Select Rows - Limit 1000”.
 - În grila de date afișată, introduceți direct în rândul nou valorile: `denumire = EduCode 2025`, `tematica = Inovatie in predarea informaticii`, `site_oficial = NULL` (sau lăsați gol).
 - Apăsați butonul „Apply” din colțul din dreapta jos pentru a executa inserarea în baza de date.

e) Interogări SQL pentru extragerea informațiilor de sinteză

Pentru a demonstra utilitatea modelului fizic propus, iată interogările SQL corespunzătoare cerințelor de sinteză din enunț:

- Numărul de festivaluri în cadrul cărora s-au desfășurat cel puțin două evenimente de tip workshop:**

```
SELECT COUNT(*) AS nr_festivaluri
FROM (
    SELECT e.id_festival
    FROM EVENTIMENT e
    JOIN TIP_EVENTIMENT t ON e.id_tip = t.id_tip
    WHERE t.denumire_tip = 'workshop'
    GROUP BY e.id_festival
    HAVING COUNT(e.id_eventiment) >= 2
) AS subquery;
```

- Tipurile de evenimente care nu s-au desfășurat în cadrul niciunui festival în ultimii doi ani:**

```
-- Varianta bazată pe subinterogare cu NOT IN
SELECT id_tip, denumire_tip
FROM TIP_EVENTIMENT
WHERE id_tip NOT IN (
    SELECT DISTINCT id_tip
    FROM EVENTIMENT
)
```

```
WHERE data_ora_inceput >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
);
```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice (Proiecte complete de Lecție)

Conform regulii de a rezolva toate variantele, sunt detaliate planurile didactice pentru ambele secvențe (A și B).

PLAN DIDACTIC PENTRU SECVENȚA A (Divide et Impera)

- **Clasa:** a XI-a (Profil real, specializarea Informatică)
- **Subiectul lecției:** Metoda de programare Divide et Impera
- **Metoda didactică principală:** **Problematizarea**
- **Mijloc de învățământ:** Tablă interactivă, platformă de tip compilator online (ex. Replit)
- **Formă de organizare:** Activitate frontală îmbinată cu activitate individuală
- **Competență specifică vizată:** 2.2. Construirea unor soluții pentru probleme simple care se rezolvă cu ajutorul metodelor de programare
- **Activitate de învățare:** Elevii primesc o situație-problemă (găsirea elementului maxim dintr-un șir neordonat folosind cât mai puține comparații globale simultane) și sunt ghidați să deducă divizarea șirului în jumătăți.

Scenariul Didactic pe Etape (ERRE)

1. Evocare (Moment organizatoric & Captarea atenției) - 5 min:

- **Activitatea profesorului:** Profesorul scrie pe tablă un vector de 8 elemente: $V = [12, 35, 2, 89, 45, 7, 90, 15]$. Întreabă cum găsim maximum în mod clasic (secvențial) și câte comparații se fac ($N - 1$).
- **Activitatea elevilor:** Elevii răspund că se parcurge vectorul de la stânga la dreapta menținând o variabilă de maxim. Pentru 8 elemente se fac 7 comparații.

2. Realizarea Sensului (Situația-Problemă & Dirijarea învățării) - 30 min:

- **Activitatea profesorului:** Introduce problema: „Cum am putea rezolva problema maximumului dacă am avea la dispoziție doi procesori separați care pot comunica la final?”. Pune elevii în situația de a împărți vectorul în doi subvectori independenți.
- **Activitatea elevilor:** Elevii propun împărțirea la jumătate: procesorul 1 caută maximum în prima jumătate $[12, 35, 2, 89]$, iar procesorul 2 în a doua jumătate $[45, 7, 90, 15]$. La sfârșit, se compară cele două rezultate parțiale.
- **Activitatea profesorului:** Formalizează metoda: subproblemele se rezolvă similar (prin re-împărțire recursivă) până când ajungem la vectori de dimensiune 1 sau 2, unde maximum este evident. Scrie pe tablă pseudocodul metodei Divide et Impera:
 1. Divizează (împarte problema în subprobleme similare).
 2. Stăpânește (rezolvă subproblemele elementare direct).
 3. Combină (unește soluțiile subproblemelor pentru a obține soluția finală).
- **Activitatea elevilor:** Elevii notează schema logică și încep scrierea subprogramului recursiv `int determinăMaxim(int st, int dr)` pe calculatoarele proprii.

3. Reflecție (Consolidare și feedback) - 10 min:

- **Activitatea profesorului:** Urmărește implementările elevilor și cere unui elev să prezinte codul C++.
- **Activitatea elevilor:** Elevul propune funcția recursivă:

```
int maxDiv(int st, int dr) {
    if (st == dr) return V[st];
    int m = (st + dr) / 2;
    int max1 = maxDiv(st, m);
    int max2 = maxDiv(m + 1, dr);
    return (max1 > max2) ? max1 : max2;
}
```

- **Activitatea profesorului:** Validează corectitudinea codului și explică mecanismul stivei de recursivitate pentru apelul inițial maxDiv(0, 7).

4. Extindere (Temă pentru acasă) - 5 min:

- **Activitatea profesorului:** Propune ca temă adaptarea algoritmului pentru a calcula suma elementelor vectorului folosind aceeași metodă.

—

PLAN DIDACTIC PENTRU SECVENȚA B (Dispozitive de stocare)

- **Clasa:** a V-a (Gimnaziu)
- **Subiectul lecției:** Dispozitive de stocare a datelor
- **Metoda didactică principală:** Mozaicul (Jigsaw)
- **Mijloc de învățământ:** Fișe de lucru diferențiate, mostre fizice de memorii (HDD demontat, SSD, Stick USB, Card SD)
- **Formă de organizare:** Lucru pe grupe colaborative
- **Competență specifică vizată:** 1.1. Utilizarea eficientă și în condiții de siguranță a dispozitivelor de calcul
- **Activitate de învățare:** Elevii vor învăța caracteristicile diverselor unități de stocare comparându-le din punct de vedere al capacității și structurii.

Scenariul Didactic pe Etape (ERRE)

1. Evocare (Constituirea grupelor) - 7 min:

- **Activitatea profesorului:** Împarte clasa în 4 „Grupe de bază” de câte 4 elevi. Împarte în fiecare grupă rolurile (expert 1, expert 2, expert 3, expert 4).
- **Activitatea elevilor:** Elevii se așază la mese conform grupelor desemnate.

2. Realizarea Sensului (Faza experților) - 25 min:

- **Activitatea profesorului:** Distribuie fișele de studiu tematice:
 - Expert 1: Hard Disk-ul (HDD) - mecanism magnetic, capacități mari, vulnerabilitate la șocuri.
 - Expert 2: Solid State Drive (SSD) - cipuri flash, viteză mare, rezistență fizică.
 - Expert 3: Stick-ul USB și Cardurile SD - portabilitate, utilizare în camere/telefoane.
 - Expert 4: Unități de măsură (Octet, KB, MB, GB, TB) și transformările de bază.

Cere experților cu același număr de la grupe diferite să se reunească în „Grupe de experți” pentru a studia materialul primit timp de 10 minute.

- **Activitatea elevilor:** Elevii colaborează în grupele de experți, examinează mostrele fizice primite de la profesor (ex. expertul 1 analizează discurile magnetice din interiorul HDD-ului deschis) și își notează ideile principale pe fișa de expert.

- **Activitatea profesorului:** Supervizează grupele de experți, oferind lămuriri. Apoi le cere să se întoarcă în grupele lor de bază și să predea pe rând colegilor ceea ce au învățat.
 - **Activitatea elevilor:** Întorși în grupele de bază, fiecare expert prezintă propria secțiune. Ceilalți membri notează caracteristicile în tabelul centralizator de pe fișa de bază.
3. **Reflecție (Evaluarea cunoștințelor) - 13 min:**
- **Activitatea profesorului:** Propune un scurt test-quiz pe calculator sau la tablă în care cere compararea dispozitivelor: „**Ce alegem pentru a porni rapid calculatorul?**”, „**Ce alegem pentru a stoca 2 TB de filme ieftin?**”.
 - **Activitatea elevilor:** Elevii răspund argumentat: SSD pentru sistemul de operare datorită vitezei mari; HDD pentru stocare ieftină de volum mare.
4. **Extindere (Temă pentru acasă) - 5 min:**
- **Activitatea profesorului:** Tema presupune analizarea calculatorului personal de acasă și identificarea tipului și capacității dispozitivului de stocare montat pe acesta.
-

2. Evaluarea rezultatelor școlare: Proba Scrisă

a) Avantaje și limite ale probei scrise:

- **Avantaje:**
 1. **Obiectivitate:** Permite aplicarea unui barem unic și stabil de corectare pentru toți elevii, reducând subiectivitatea evaluatorului.
 2. **Comparabilitate:** Rezultatele pot fi comparate direct deoarece toți elevii rezolvă aceleași sarcini în aceleași condiții de timp și spațiu.
 3. **Gestionarea timpului:** Permite evaluarea simultană a unui număr mare de elevi într-un interval fix de timp.
- **Limită:**
 - Nu permite evaluarea directă a competențelor practice și de manipulare fizică a echipamentelor sau utilizarea directă a unui mediu de programare pe calculator (depanare în timp real, erori de compilare active).

b) Elaborarea itemilor de evaluare

PENTRU SECVENȚA A (Subprograme / Divide et Impera)

- **Opțiunea A1: Item Obiectiv (Alegere Multiplă)**
 - **Tip:** Alegere multiplă cu un singur răspuns corect.
 - **Enunț:** Fie metoda Divide et Impera. Care este numărul total de apeluri ale funcției recursive `maxDiv(st, dr)` (inclusiv apelul inițial) realizate pentru determinarea maximului dintr-un vector cu 4 elemente? a) 3
b) 5
c) 7
d) 4
 - **Răspuns corect:** c) 7.
 - **Barem/Justificare:** Arborele de apeluri recursivi este: `maxDiv(0,3)` care apelează `maxDiv(0,1)` și `maxDiv(2,3)`. `maxDiv(0,1)` apelează nodurile frunză `maxDiv(0,0)` și `maxDiv(1,1)`. `maxDiv(2,3)` apelează `maxDiv(2,2)` și `maxDiv(3,3)`. În total avem $1 + 2 + 4 = 7$ apeluri.

• **Opțiunea A2: Item Subiectiv (Rezolvare de probleme)**

- **Tip:** Item cu răspuns construit extins.
- **Enunț:** Scrieți o funcție recursivă în C++ sau Pascal care, folosind metoda Divide et Impera, determină dacă toate elementele unui vector sunt numere pare. Funcția va returna `true` (sau 1) dacă toate sunt pare, respectiv `false` (sau 0) în caz contrar.
- **Răspuns așteptat (C++):**

```
bool toatePare(int V[], int st, int dr) {
    if (st == dr) return (V[st] % 2 == 0);
    int m = (st + dr) / 2;
    return toatePare(V, st, m) && toatePare(V, m + 1, dr);
}
```

PENTRU SECVENȚA B (Dispozitive de stocare)

• **Opțiunea B1: Item Semiobiectiv (De completare)**

- **Tip:** Item de completare.
- **Enunț:** Completați spațiul liber din enunțul următor astfel încât acesta să fie corect din punct de vedere științific: „Dispozitivul de stocare intern caracterizat prin absența pieselor mecanice în mișcare și viteze foarte mari de acces la date se numește _____.”
- **Răspuns așteptat: SSD (sau Solid State Drive).**

• **Opțiunea B2: Item Semiobiectiv (Răspuns scurt)**

- **Tip:** Item cu răspuns scurt.
- **Enunț:** Ordonăți descrescător următoarele unități de stocare în funcție de capacitatea lor maximă teoretică: 1.5 TB, 2048 GB, 512000 MB.
- **Răspuns așteptat: 2048 GB** (care este egal cu 2 TB), urmat de **1.5 TB**, urmat de **512000 MB** (care este egal cu aproximativ 0.5 TB).
- **Barem de notare:** Se transformă toate valorile în aceeași unitate (ex. GB):
 - 1.5 TB = 1536 GB
 - 2048 GB
 - 512000 MB = 500 GB.

Ordinea descrescătoare corectă este: 2048 GB > 1.5 TB > 512000 MB.